

Weekly report — fixed-income pricing module

Date: 2026-08-30 · **Covers:** the six rows you marked on the `Pivot of Corp Bonds` sheet

You went down the coupon-type pivot with us and marked six rows **no** — the ones the restructured code did not yet cover. All six are now covered, and this report answers them cell by cell.

One of the six turned out not to be a data problem at all. It was a bug in how we read a market-data file, and it was also quietly costing us a bond in the row you had already marked **finished**. That is section 3, and it is the most important thing here.

As before: everything except **section 6** is written to be read without the code. Section 6 is for the engineering team and the finance reader can skip it.

1. Your six cells, answered

A word on the counts first, because several different denominators are easy to confuse. The pivot counts **rows on the Corporate Bonds sheet** (676 of them). That sheet lists some securities more than once — 60 of them — so 676 rows are 616 distinct bonds. Each row below therefore shows the pivot count, the number of distinct securities behind it, how many are live holdings, and how many now produce a full set of numbers.

Cell	Coupon type	Pivot rows	Securities	Held	Priced	Not priced
F12	Fixed → Floating	5	5	5	4	1
F13	7.00% before 01-Mar-2006, 7.50% after	2	1	1	1	—
F14	GBP LIBOR + Spread	1	1	1	1	—
F15	Reference Rate + Spread	12	12	12	11	1
F16	EURIBOR + Spread	9	9	9	5	4
F20	Step-up schedule	1	1	1	1	—
	total	30	29	29	23	6

A correction to how we put this to you. An earlier draft of this table showed F13 as "2 rows, 1 held", which reads as though one of those two is not a holding. It is not: your sheet lists the *same* bond twice. One bond, held, priced. Every one of the 29 distinct securities behind your six cells is a live holding — none of them is missing from the book.

The six that do not price are not a modelling gap. Five are bonds that pay a fixed coupon for some years and then switch to a floating one, and the *margin they pay after the switch* is not in the workbook and is not public — it is on the Bloomberg list already with you. Each is carried at the custodian's price with that reason attached, and each becomes a priced bond the moment the margin arrives: it is one cell in a data file, with no code change at all. The sixth is a defaulted bond, which is carried at its recovery mark

on purpose — a spread calculated against a borrower who has stopped paying would be a number without a meaning.

We also, for free, covered the six rows immediately above yours (**Fixed** → **Reset** , rows 6–11), because they are the same instrument shape and share the same engine. Three of those six now price; the other three are waiting on the same missing margins.

2. What these coupon types actually are

Four families sit behind your six cells, and they need genuinely different treatment:

- **Floating-rate note** (F14, F15, F16) — the coupon is not fixed at all. It resets periodically to a market interest rate plus a contractual margin. The consequence worth knowing: such a bond keeps re-setting its own interest-rate risk away, so a thirty-year floating note carries the rate risk of **a single coupon period**, not of thirty years.

Two cautions on reading those risk figures, because the usual shorthand — "a floater's risk is the time to its next reset" — is only true in one of three cases.

First, whether it is the time to the **next** reset or the time **since the last** one depends on whether we know the coupon that is currently running. If we do, the figure is the time forward to the next reset; if that coupon has to be estimated from the market curve, the figure is the same length of time but pointing **backwards**, and therefore negative. Both are correct and both are less than one coupon period. *Second*, a bond trading well below face value carries an additional, genuinely negative figure that grows with maturity — that is the discount itself unwinding, not a sign error. The module reports the next reset date beside every risk figure so the two can be checked against each other, and section 6 gives the exact numbers.

A third caution, about the **spread** rather than the risk. Most of these notes are described in the workbook only as "... + Spread", with no number: the contractual margin is not recorded anywhere we can read. For those, the spread we report absorbs that unknown margin **as well as** the borrower's credit, so it should be read as a total discount margin and not as a clean credit spread. Where a margin is documented, it is used and the spread is a credit spread. The output says which of the two it is for every bond, rather than leaving it to be assumed. - **Fixed** → **Floating** (F12) — fixed for some years, then floating. Priced as one bond with one credit spread across both halves, because it is one borrower's one promise. Splitting the spread in two would be inventing a second borrower. - **Stepped / step-up** (F13, F20) — the coupon changes over time on a schedule that is **known in advance**. There is no option and no uncertainty here, so these are not a new model at all: they are ordinary bonds priced with a table of coupons instead of one coupon. Saying so is worth more than building something. - **Zero / structured** (F21, which you had already annotated "just corp bond (fixed)") — agreed, and that is how it is treated.

3. F14 was our bug, not missing data — please drop that request

We have been telling you the single British-pound bond could not be priced because our UK interest-rate curve "was not arbitrage-free". That was wrong, and we should correct it plainly.

The market-data files we were given are not all written the same way. Most store interest rates as decimals — **0.0304** meaning 3.04%. The **British-pound file stores them as percentages** — **3.04** meaning 3.04%. Our loader assumed decimals for every file, so it multiplied the pound rates by a hundred and

produced a curve with three-year rates near 200%. The bootstrapping step then, correctly, refused to build a curve out of it, and reported the only thing it could see: that the curve was not arbitrage-free. We read that as a statement about the data, and raised a request for a replacement UK curve.

The curve was always fine. Read correctly, the 31 March 2009 file gives 1.18% at two years, 2.34% at five, 3.16% at ten and 4.16% at thirty — the actual gilt market that day.

What this changes:

	Before this fix	After it
France Télécom 7.50% of 2011 (GBP) — your F14	not priced	spread 205.3 bp , rate sensitivity 1.86 years
A UK 5.50% bond of 2033 (GBP)	missing from the output entirely	spread 197.3 bp , rate sensitivity 12.55 years
Corporate bonds in the output	564	565
Of those, fully priced	553	555

(A later review this week found one more bond in the same condition and raised the output to 566; section 4.5 has that one.)

The second bond is the part that matters beyond this week. It was not flagged, it was silently **skipped** — and it is a plain fixed-coupon bond, which is to say it belongs to the row you had already marked *finished*. A completeness count can be wrong in a direction that no report shows you, and this one was.

Two independent checks that the new numbers are right. The bond's spread measured against the *dollar* curve is 279.9 bp and against its own *sterling* curve 197.3 bp; the 83 bp gap is precisely the difference between gilt and Treasury yields at that maturity. And the France Télécom spread of 205 bp sits exactly where a single-A rated telecom belonged in March 2009, next to its own euro-denominated bonds.

We also checked every one of the 26 market-data files this way, at three dates each. One other file — Danish krone — has the same percentage convention. It is not used by this portfolio, but it is now declared, so the next person to reach for it does not repeat this. And the loader now **refuses** any file whose rates come out above 100%, naming the units as the cause, so this class of mistake cannot again disguise itself as a statement about the market.

Action for you: the request for a replacement UK curve can be withdrawn. Nothing else on the outstanding Bloomberg list changes.

3.1 A second one of the same kind — caught before it reached you

Having found one bond that had gone missing without a trace, we went looking for others of the same shape. There was one, and this time we found it **before** it affected any number we have shown you.

Five bonds in the portfolio are callable — the borrower may repay early. Three are priced on the option model, and one is waiting for its call terms, which are on the outstanding data request. That accounts for four. **The fifth had fallen between two rules.** One part of the code treated a call less than a week before maturity as economically irrelevant and priced the bond normally; another part only sent bonds to the option model when the call was more than a year before maturity. A bond whose call sits 90 days before

maturity satisfied neither, so it was priced by nothing — and, unlike a flagged bond, it produced no message anywhere. It is a real holding: 850,000 nominal, marked at 85.12.

It had even been written down, once, in our own work log, as a "minor loose end" — and then it fell out of every count that followed. That is the lesson rather than the bond: a completeness figure can be wrong in a direction no report shows you, and the same failure had now happened twice.

Two things changed, neither of which moves a price:

- **the counting is now mechanical.** Every bond in the portfolio must leave the calculation either with a number or with a named reason. That is checked as a set — not as a total, because a total of "3 priced + 2 skipped = 5" balances perfectly even with the wrong bond in the wrong place, which is exactly how this one stayed hidden. The check runs on every production run and stops it if any bond is unaccounted for.
- **the two rules became one.** Deciding *whether* a bond is callable now happens in one place only; the option model simply prices everything it is sent.

And a third thing, which is the part worth knowing about the model itself. When we sent that bond to the option model to see what it was worth, it returned a value for the early-repayment right of **exactly zero** — which looks like "the right is worthless". It was not. The option model makes its decisions on the bond's coupon dates, and this bond's call date falls in the final three months, after the last coupon. There was no date on which the model could consider it, so it never did. The number was not an answer; it was the absence of a question.

The model now **refuses** such a bond rather than pricing it as an ordinary one — which is why the fifth callable bond is still not priced, and is now named and explained instead. We checked all eight bonds that carry call terms today: none of them is affected, so no number you have been given is wrong. Deciding what that bond should ultimately be worth needs either a change to the option model's calendar or a documented rule about very short call windows; both are real decisions and we have not taken either quietly.

4. One change to the Excel connection

The pricing engine now accepts **seven kinds of bond** by name — plain, stepped, floating, fixed-then-floating, callable, puttable and sinking-fund — through the same single entry point, the same request and answer format, and the same error reporting as before.

Three different numbers matter here, and it is worth being exact about which is which, because saying "the spreadsheet can ask for any of seven" would overstate what we have actually put in front of you:

	how many	which
the engine prices, and the request format covers	7	all of the above
the spreadsheet can currently build a request for	5	all except stepped and fixed-then-floating — no cells exist yet for a coupon table, a margin or a switch date
we have tested end-to-end from real Excel	4	plain, callable, puttable, sinking-fund

The gap is a matter of cells on a sheet, not of engineering: adding the missing four cells and one small table is straightforward, and we have deliberately left it until you have told us what the sheet should look like (section 8). One consequence worth flagging: a floating note sent from Excel today has nowhere to carry its margin, so the spread that comes back absorbs the margin rather than isolating credit. From the engine directly, it does not.

We made the engine change **once**, covering all seven, rather than twice. A request that does not name a type is still treated as a plain bond, so **nothing that works today stops working** — including the spreadsheet as it stands, whose original 23 automated Excel checks all still pass untouched.

Each type reports the one or two extra numbers only it has: the next reset date for a floating note, the switch date for a fixed-then-floating one, and — for the three types where somebody can repay or return the bond early — the volatility answer from your last round, in both directions.

There is one thing the module will now refuse rather than answer. If a fixed-then-floating bond is sent without its post-switch margin, we do not price it with a zero. A guessed margin produces a confident-looking number for a bond that is half-modelled, and nothing in the output would say so.

4.1 Additional integration follow-through — embedded-option bonds through Excel

Separate from your six cells, and reported separately on purpose: this is follow-through on the interface promise in the last report, not part of the floating-rate work.

The spreadsheet can now send the three bond types with early-repayment rights — callable, puttable and sinking-fund — and show their results. This was already true of the engine; what was missing was the spreadsheet side, and it is now built and tested on real Excel.

Exercise schedules are entered as ordinary Excel **tables**, one row per date:

```
FIP_CallSchedule      Date | Price per 100
FIP_PutSchedule       Date | Price per 100
FIP_SinkingSchedule   Date | Fraction of the amount outstanding | Price per 100
```

Tables rather than fixed blocks of cells, for two practical reasons: a schedule can be any length, and a table can sit on any sheet — so none of this commits you to a layout before you have chosen one.

Two behaviours are worth knowing because they are deliberate:

- **A blank row is ignored; a half-filled row is refused**, in Excel, before anything is sent, naming the table and the row number. We do not fill in a missing exercise price or redemption fraction. Those are contractual terms, and a plausible guess is worse than a stop.
- **Your existing sheet is untouched**. A request that does not name a bond type is treated exactly as it was before, which is why every one of the original spreadsheet checks still passes without modification.

Your volatility question, now answerable from the spreadsheet. Changing the volatility cell on a callable bond and re-running gives, on our test bond, a spread of 639.58, 635.78 and 629.83 basis points at 10%, 15% and 20% volatility — the same tightening the last report described, now driven from Excel rather than from Python. That is three ordinary runs, not a special feature.

One caveat we want stated rather than buried. The portfolio holds callable bonds, so that type is tested against real holdings. It holds **no** puttable and **no** sinking-fund bonds — those two are tested on invented examples. Those tests prove the model and the connection behave correctly; they prove nothing about your portfolio, and we have labelled them that way everywhere they appear.

The automated spreadsheet checks went from 23 to **50**.

What is still not built is the layout — the daily-use sheet, whether that is one sheet with a type dropdown or a small sheet per type. That is the question in section 8, and it is genuinely yours to answer: what exists today is an engineering test surface, not a sheet we would ask anyone to work in. The two bond types the sheet cannot yet send are waiting on the same answer.

4.5 A review pass over everything above — three corrections

Before sending this we reviewed the week's work against the live outputs rather than against our own notes. Three things came out of it. All are corrections to work described earlier in this report, and we would rather you saw them here than found them later.

One more bond that was invisible — the output is 566, not 565

This is the **third** instance of the same shape as sections 3 and 3.1, and it is the largest position of the three: **8.78 million nominal**, held across three lots.

The word "defaulted" was doing two jobs in our code. It is a description of a bond's *coupon* — your workbook records some as "N/A (Defaulted)" — and it is separately a *credit rating* of D. A bond can have one without the other, and this one does: its rating is in default, while its coupon formula is an ordinary fixed rate. Two different pieces of code handled defaulted bonds, one keyed on each meaning, and a bond with this combination matched neither. It is now handled **once, by the rating**, and appears as a named recovery line at your custodian's mark — we do not compute a spread for a defaulted bond, because the number would describe expected recovery rather than credit.

A fourth defaulted holding stays out of the output, correctly, but its stated reason was also wrong: it was reported as excluded for being in default, when what actually excludes it is its coupon type, one of the categories you told us in July to leave out permanently. It now says so.

Floating-rate risk: one number per bond, not two answers depending on a data field

The coupon a floating-rate note is paying *right now* was set at its last reset date, in the past. It is a known amount, and a change in today's interest rates cannot alter it.

Our model held it fixed whenever your file recorded the number, and re-estimated it from the curve whenever your file left it blank — and in that second case, moving rates moved a coupon that had already been decided. The effect was not small: it reversed the sign of the reported rate sensitivity. Two notes with the same economics could receive opposite-signed answers, and which one you got depended on whether a field in the custodian file happened to be filled in.

Both cases now hold the coupon fixed. Six of the seven floating notes moved, all in the same direction, and **no price and no spread changed at all** — the correction touches only the rate-sensitivity columns:

bond	market price	sensitivity before	after
TNTD04955876	69.62	-1.13	-0.43
TNTD04259874	72.64	-1.37	-0.69
TNTD03035014	92.92	-0.33	+0.20
TNTD04882955	67.04	-1.85	-1.48
TNTD04131505	81.34	-0.47	-0.17
TNTD03027773	98.54	-0.20	+0.05

Each move equals one coupon period scaled by how far below par the bond trades, to within 3% — which is the size the correction should be, and is how we checked it rather than eyeballing.

Four remain negative, and that is genuine rather than left-over error. A floating note trading in the 60s does so because its credit spread is wide, and a wide spread behaves like a fixed annuity sitting on top of the floating coupon, carrying ordinary fixed-income sensitivity. The two notes closest to par cross into small positive numbers, which is the textbook result for a floater and is the clearest sign the fix does what it should.

Numbers now say what they rest on

Two things were true, documented, and invisible at the point where somebody reads a number.

No call schedule in this project has been confirmed. All nine are a date from your custodian file with a repayment price of 100 assumed on top — a convention you approved for version 1, not a term anyone has read out of a document. A "100.00" in a spreadsheet cell looks the same either way. Every affected row now carries `provisional`, names where the terms came from, and dates them; the count of confirmed exercise terms in this project is **zero**, and the output says so. These are on the existing confirmation-only list and we are **not** asking you for them now — every one of those bonds prices today.

Six of the seven floating notes are priced on an estimated current coupon (the fix above), and those rows now say `base_curve_proxy` and mark their sensitivities provisional.

Neither label changes any number. They describe the inputs, and we test that a labelled result equals an unlabelled one exactly.

5. What we deliberately did not do

- **We did not design the daily-use worksheet.** The engine takes every bond type and the spreadsheet can send five of them (section 4), but putting seven on one sheet — or on seven small sheets — is a layout question we would rather settle with you than guess at. What exists is an engineering test surface, clearly labelled as such.
- **No bond was re-classified to make a count look better.** The six unpriced bonds stay unpriced and named.
- **We did not touch the holdings workbook.** Your column F is your marking; section 1 is the evidence for updating it, and the update is yours to make.

6. For the engineering team

Structure. Three engines moved into the template layout this week — `core/pricing/floating.py`, `core/pricing/hybrid.py` and `core/pricing/coupon_schedule.py` — each a verbatim move with the old import path left as a compatibility shim. Three thin per-product wrappers (`assets/corporate/{floating,hybrid,stepped}.py`) provide the one-simple-function-per-output surface, with numbered input blocks in every docstring.

The move also closed a layering violation: `core/pricing/cashflows.py` had been importing from the legacy `pricing` package, i.e. the core reaching upward into a not-yet-migrated layer. It now imports a sibling.

Interface. `analyze_payload(request) → response`, dispatching on `bond.instrument_type`. Adding an engine later is one map entry and one function — no new endpoint, no second envelope, no second error vocabulary. `analyze_vanilla_payload` is retained as an alias.

Verification.

Automated checks	390 , in about 35 seconds (223 when this round began)
Population accounting	enforced at run time — every bond leaves with a number or a named reason, checked as a SET of identifiers, not as a total
Production outputs after each migration step	byte-identical to a pre-change baseline, all five driver files
Endpoint vs direct function call	asserted equal with <code>==</code> , per instrument type, not a tolerance
Compatibility shims	asserted to re-export the <i>same object</i> , not an equivalent one

Two details worth knowing before you run it:

- **Cross-platform determinism has a limit worth writing down.** Full 566-bond driver outputs from Windows and from the Linux server differ by up to **3.6e-8 relative**, and entirely in the convexity column — a second difference divided by the square of a one-basis-point bump amplifies a last-bit rounding difference by 10^8 . Prices, spreads and durations agree to about $1e-12$, and every text column is identical. A byte-for-byte comparison across platforms will therefore show a difference that is not a regression; compare on one platform, or compare with a tolerance.

- **The floating-rate duration used to have two regimes with opposite signs. It now has one.** An earlier draft of this report described the two as both correct, differing only in interpretation. On review that was wrong, and section 4.5 explains what changed: the coupon a floating note is *currently* paying was fixed at its last reset, so a move in today's interest rates cannot change it. We were holding it fixed when your file recorded it and re-estimating it when your file did not, which is what produced the second sign. Both cases now hold it fixed and both give *plus* the time to the next reset.

```
python -m pytest -q                                     390 checks, ~35 s
python scripts/price_json.py --input request.json --output response.json
powershell -File integrations/excel_vba/tests/Run-BridgeTests.ps1 48 Excel checks
powershell -File ...\\Run-BridgeTests.ps1 -PythonExe <python.exe> 50, against a live engine
```

7. Next

- **Mortgage-backed securities** remain the largest outstanding block, and they are waiting on the pool data pull, not on us — the engine skeleton is built to the exact field list.
- **The demonstration spreadsheet for the new bond types**, once you tell us how you would like them laid out.

8. One question for you

Do you want the extra bond types on the demonstration sheet, and if so, in what shape? One sheet with a bond-type dropdown that shows and hides the fields each type needs, or a separate small sheet per type? We have no preference and it is a couple of hours either way — but it is your team's daily view, so it should be your call.